CI/CD with Azure DevOps

`azure-pipelines.yml` (repo root) runs the Release steps of the runbook automatically:

Stage	Runs on	What it does
CI	every push to `main` and every pull request into `main`	Frontend: `npm ci`, `oxlint`, `tsc -b && vite build`. Backend: `alembic upgrade head` on a throwaway Postgres 16, then the whole `pytest` suite, including the PostgreSQL RLS tests and the Redis tests. Test results appear on the run's **Tests** tab
Build	`main` only	`az acr build` for `fddt-backend` and `fddt-web`, tagged with the first 7 characters of the commit SHA
Deploy	`main` only, **after approval**	Prints the PITR target time and the previous image, runs the `fddt-migrate` job and waits for *Succeeded*, updates `fddt-api`, the three workers, `fddt-beat` and `fddt-web`, then checks `https://<APP_HOST>/api/health`

The pipeline **updates** existing apps. It does not create them, so do the first deployment with the runbook (steps 1–22). After that, every merge to `main` is released by the pipeline. You no longer need Cloud Shell for releases.

Nothing secret goes into Azure DevOps. The apps keep reading their secrets from Key Vault, and the pipeline signs in to Azure without a password, through workload identity federation.

What you need to fill in

Collect these values before you start. Run the commands in Cloud Shell after `source ~/fddt-deploy/reload.sh`.

Value	Where it goes	How to get it
Subscription ID	Service connection	`az account show --query id -o tsv`
Subscription name	Service connection	`az account show --query name -o tsv`
Tenant ID	Service connection	`az account show --query tenantId -o tsv`
Client ID of the pipeline identity	Service connection (option B)	Created in step 2 below: `az identity show -g $RG -n $PREFIX-cicd --query clientId -o tsv`
`RG`	Variable group	`echo $RG` (e.g. `rg-fddtprod`)
`ACR`	Variable group	`echo $ACR`: the registry **name** (e.g. `fddtprodacr`), not the `.azurecr.io` address
`APP_HOST`	Variable group	The address users open, **without** `https://`. With a custom domain: `echo $APP_DOMAIN`. Sandbox without one: `az containerapp show -g $RG -n fddt-web --query properties.configuration.ingress.fqdn -o tsv`

---

1. Project, repository and agents

1. In **https://dev.azure.com/<org>**, create a project (or use the client's), then **Repos › Files › Import repository**, or push to the empty repo:

   ```
   git remote add azure https://dev.azure.com/<org>/<project>/_git/FDDT
   git push azure main
   ```

2. **Check that you can run builds.** Go to **Organization settings › Pipelines › Parallel jobs**. *Microsoft-hosted* must show at least **1**. New organizations often show **0**. In that case:

   - request the free grant at **https://aka.ms/azpipelines-parallelism-request** (takes 2–3 business days), or
   - buy one parallel job (**Organization settings › Billing**, ~$40/month), or
   - run a self-hosted Linux agent with Docker installed and set `pool: name: <your pool>` in the YAML.

   Without one of these, every run fails with *No hosted parallelism has been purchased or granted*.

2. Service connection `fddt-azure`

The pipeline signs in to Azure with this connection. The name **must be exactly** `fddt-azure`. Otherwise, change `serviceConnection` at the top of `azure-pipelines.yml`.

The identity needs **Contributor on the resource group** and nothing else. Contributor covers `az acr build`, `az containerapp update` and starting the migration job. It does not need Key Vault access.

Option A: automatic (needs rights in Entra ID)

Project settings › Service connections › New service connection › Azure Resource Manager → *Identity type*: **App registration (automatic)** → *Credential*: **Workload identity federation** → *Scope level*: **Subscription** → pick the subscription and the resource group `rg-fddtprod` → *Service connection name*: `fddt-azure` → leave *Grant access permission to all pipelines* unticked → **Save**.

This fails if your account cannot create app registrations in the client's Entra ID, or cannot assign roles on the resource group. Sandbox accounts usually can't do either. If it fails, use option B.

Option B: managed identity (works in most sandboxes)

The client's admin, or anyone with **Owner** or **User Access Administrator** on the resource group, runs this in Cloud Shell:

```
source ~/fddt-deploy/reload.sh
az identity create -g $RG -n $PREFIX-cicd -o none
CICD_CLIENT_ID=$(az identity show -g $RG -n $PREFIX-cicd --query clientId -o tsv)
CICD_PRINCIPAL=$(az identity show -g $RG -n $PREFIX-cicd --query principalId -o tsv)
az role assignment create --assignee-object-id $CICD_PRINCIPAL --assignee-principal-type ServicePrincipal \
  --role Contributor --scope $(az group show -n $RG --query id -o tsv)
echo "Client ID: $CICD_CLIENT_ID"
echo "Tenant ID: $(az account show --query tenantId -o tsv)"
echo "Subscription: $(az account show --query id -o tsv)  $(az account show --query name -o tsv)"
```

Then in Azure DevOps:

1. **Project settings › Service connections › New service connection › Azure Resource Manager** → *Identity type*: **Managed identity** (on older UIs: **Workload identity federation (manual)**) → **Next**.
2. *Service connection name*: `fddt-azure`. The next page shows an **Issuer** and a **Subject identifier**. Copy both and leave the page open.

3. Back in Cloud Shell, create the federated credential on the identity with those two values:

```
az identity federated-credential create -g $RG --identity-name $PREFIX-cicd -n azure-devops \
  --issuer "<Issuer from Azure DevOps>" --subject "<Subject identifier from Azure DevOps>" \
  --audiences api://AzureADTokenExchange
```

4. Fill in the rest of the form: *Subscription ID*, *Subscription name*, *Tenant ID*, and **Client ID** = the identity's client ID from above. Tick **Verify** if it's offered, then **Save**.

✅ **Verify:** open the connection. *Edit › Verify* says *Verification succeeded*. If it says *AADSTS70021: No matching federated identity record*, the issuer or subject was copied wrong. Delete and re-create the federated credential.

3. Variable group `fddt-production`

Pipelines › Library › + Variable group → name `fddt-production` → add:

Name	Example
`RG`	`rg-fddtprod`
`ACR`	`fddtprodacr`
`APP_HOST`	`fddt.client.com` or `fddt-web.<hash>.uaenorth.azurecontainerapps.io`

None of these is secret, so don't lock them. **Save**.

4. Environment `fddt-production` (approval gate)

Pipelines › Environments › New environment → name `fddt-production` → *Resource*: **None** → **Create**. Then ⋮ › **Approvals and checks**:

- **+ › Approvals**: add the people who approve releases. Only they can let the Deploy stage start.
- **+ › Exclusive lock**: two releases never run at the same time. Set *Lock behavior* to **sequential**.

5. Create the pipeline

Pipelines › New pipeline › Azure Repos Git → the FDDT repo → **Existing Azure Pipelines YAML file** → branch `main`, path `/azure-pipelines.yml` → **Run**.

On the first run, Azure DevOps pauses and asks to **Permit** the pipeline to use the service connection, the variable group and the environment. Click **View › Permit** for each. It asks only once.

6. Run CI on pull requests

Azure Repos ignores the `pr:` block in the YAML. Pull requests are validated by a branch policy instead: **Repos › Branches** › `main` › ⋮ › **Branch policies › Build validation › +** → pick this pipeline → *Trigger*: Automatic, *Policy requirement*: **Required** → **Save**. A PR can then merge only if CI passed. Build and Deploy are skipped for PRs.

Day-to-day

- **Release:** merge to `main` → CI → Build → the approvers get an email → **Review › Approve** → Deploy.

- **Check a release:** the Deploy log prints the migration execution name, each status poll, `Deployed <tag>` and the `/api/health` answer. In the portal, each app's **Application › Revisions and replicas** shows the new revision.
- **Rollback without a migration:** open the last good run → **Deploy** stage → **Rerun stage**. That run's commit is redeployed (its images are still in ACR).
- **Rollback when the release had a migration:** don't rerun an older Deploy. The old image can't run `alembic upgrade head` against a newer database. Follow the runbook's Rollback. The Deploy log shows the **PITR target** time.
- **Docs-only commits** (`docs/**` , `*.md`) don't trigger the pipeline.

Troubleshooting

Error	Cause / fix
No hosted parallelism has been purchased or granted	See step 1.2
The pipeline is not valid. … service connection fddt-azure could not be found	The connection is named differently, or the pipeline was not permitted to use it (step 5)
AuthorizationFailed … does not have authorization to perform action	The Contributor role is missing on the RG, or was assigned less than ~5 minutes ago
`az acr build` : *The resource with name '…' … could not be found*	`ACR` holds the login server instead of the registry name, or the wrong `RG`
Migration `Failed`	The apps were not touched. Open **fddt-migrate › Execution history** → the execution → **Console logs**. Runbook step 13 lists the usual errors
Smoke test fails but the apps are *Running*	`APP_HOST` is wrong (it must have no `https://`), or the custom domain/TLS isn't bound yet: use the `*.azurecontainerapps.io` FQDN
CI `pytest` fails only in the pipeline	Run the same suite locally against Postgres and Redis: local-setup.md › Tests